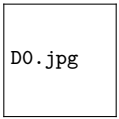
Database Management Systems

Lecture 12: Relational Algebra III — Division and Query Equivalence

Dr. Innocent Nyalala



IIT Madras Zanzibar

27 March 2026

Outline

Division: $R \div S$ — The "For All" Operator

Division $\div$

$R \div S$ returns all values of attributes in R that are **associated with every** value in S.

Use case: "Find all students who are enrolled in *every* required course."

Formal definition: Let $R(X, Y)$ and $S(Y)$. $R \div S = \{t[X] \mid \forall s \in S,\ \exists r \in R \text{ with } r[X] = t[X] \text{ and } r[Y] = s[Y]\}$

SQL Equivalent

Division has no direct SQL operator. It is expressed using NOT EXISTS or double negation:

Enrolment(SID, CourseID):

SID	CourseID
S01	C101
S01	C102
S02	C101
S03	C101
S03	C102

Required(CourseID):

CourseID
C101
C102

Enrolment $\div$ Required:

SID
S01
S03

S01 and S03 enrolled in BOTH C101 and C102.
S02 is in C101 only — excluded.

Computing Division Using Fundamental Operations

Division from Primitives

$R(X, Y) \div S(Y)$ can be computed as:

$$T_1 = \pi_X(R)$$
$$T_2 = \pi_X\big((T_1 \times S) - R\big)$$
$$R \div S = T_1 - T_2$$

Intuition:

T_1 = all X values that appear in R

$T_1 \times S$ = all (X,Y) combinations that *should* exist

$(T_1 \times S) - R$ = combinations that are *missing* in R

T_2 = X values with at least one missing Y

$T_1 - T_2$ = X values with no missing Y (i.e., all

Step-by-step trace with our example:

$T_1 = \pi_{\text{SID}}(\text{Enrolment}) = \{S01, S02, S03\}$

$T_1 \times \text{Required} = \{$
(S01,C101), (S01,C102),
(S02,C101), (S02,C102),
(S03,C101), (S03,C102)$\}$

$(T_1 \times \text{Required}) - \text{Enrolment} =$
$\{(S02, C102)\}$

$T_2 = \pi_{\text{SID}}(\ldots) = \{S02\}$

$T_1 - T_2 = \{S01, S03\}$ ✔

Aggregate Functions: $\mathcal{G}$

Aggregate Operation $\mathcal{G}$

Standard relational algebra has no aggregation. We add the $\mathcal{G}$ operator:

$$_{G_1, G_2, \ldots, G_n}\mathcal{G}_{F_1(A_1), \ldots, F_m(A_m)}(R)$$

- G_i: grouping attributes (GROUP BY columns)
- F_j: aggregate functions (SUM, COUNT, AVG, MAX, MIN)
- A_j: attributes to aggregate

SQL Correspondence

```
-- Count per department SELECT Dept,
COUNT(SID) AS Total FROM Student GROUP BY Dept;
```

Not Standard

The $\mathcal{G}$ operator is an **extension** to core relational algebra. Codd's original 6 operations do not include aggregation. SQL adds it through GROUP BY.

Example:

Query Equivalence: Optimiser Transformations

Equivalent Expressions

Two relational algebra expressions are **equivalent** if they produce the same result on every valid database instance.

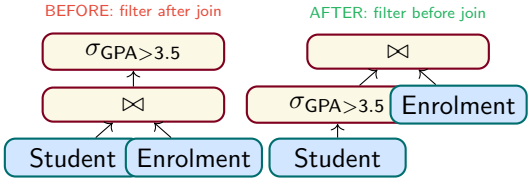
Key equivalence rules the optimizer uses:

1. **Cascade selection:**
   $\sigma_{c_1 \wedge c_2}(R) \equiv \sigma_{c_1}(\sigma_{c_2}(R))$
2. **Commutativity of selection:**
   $\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$
3. **Push selection down past join (predicate pushdown):**
   $\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S$ (if c involves only R)

Why This Matters for SQL

These rules let the optimizer reorder operations to reduce intermediate result sizes.

Predicate pushdown is the most impactful: apply σ before $\bowtie$ to reduce the number of tuples flowing into the join.



ICA Activity: Algebra for 5 Queries

In-Class Activity (ICA) — 10 min

Schema: Student(SID, Name, Dept, GPA), Enrolment(SID, CourseID, Grade), Course(CourseID, Title, Credits)

Write a relational algebra expression for each:

1. Names of all students in the CS department
2. Course titles of courses worth 3 or more credits
3. Names of students who have a grade of 'A' in any course
4. Names of students who are enrolled in the course titled 'Databases'
5. Student IDs of students who are enrolled in **every** 3-credit course (use division)

Submit your answers at the end of class.

Lecture Summary

Key Takeaways

1. Division $\div$ answers "for all" queries: find X values associated with every Y value
2. Division can be expressed using π, $\times$, and $-$ (fundamental ops)
3. Aggregate operator $\mathcal{G}$ extends algebra with GROUP BY / aggregation
4. Equivalence rules let the optimizer reorder operations safely
5. Predicate pushdown is the single most impactful optimization

Next Lecture (L13)

Tuple Relational Calculus

- TRC syntax: $\{t \mid P(t)\}$
- Safe expressions
- TRC $\equiv$ relational algebra (Codd's theorem)
- QBE visual language

Lab (T4 — this week)

Algebra Drill: translate 15 English queries to algebra expressions. Verify results in SQL.